LOMBA KOMPETENSI SISWA (LKS) SEKOLAH MENENGAH KEJURUAN TINGKAT KOTA BEKASI - 2026

NASKAH SOAL

Bidang Lomba
Cloud Computing

Topic 2 - DynamoDB, Lambda-Based Log Processing, EventBridge Scheduling, and Visit Count API Development

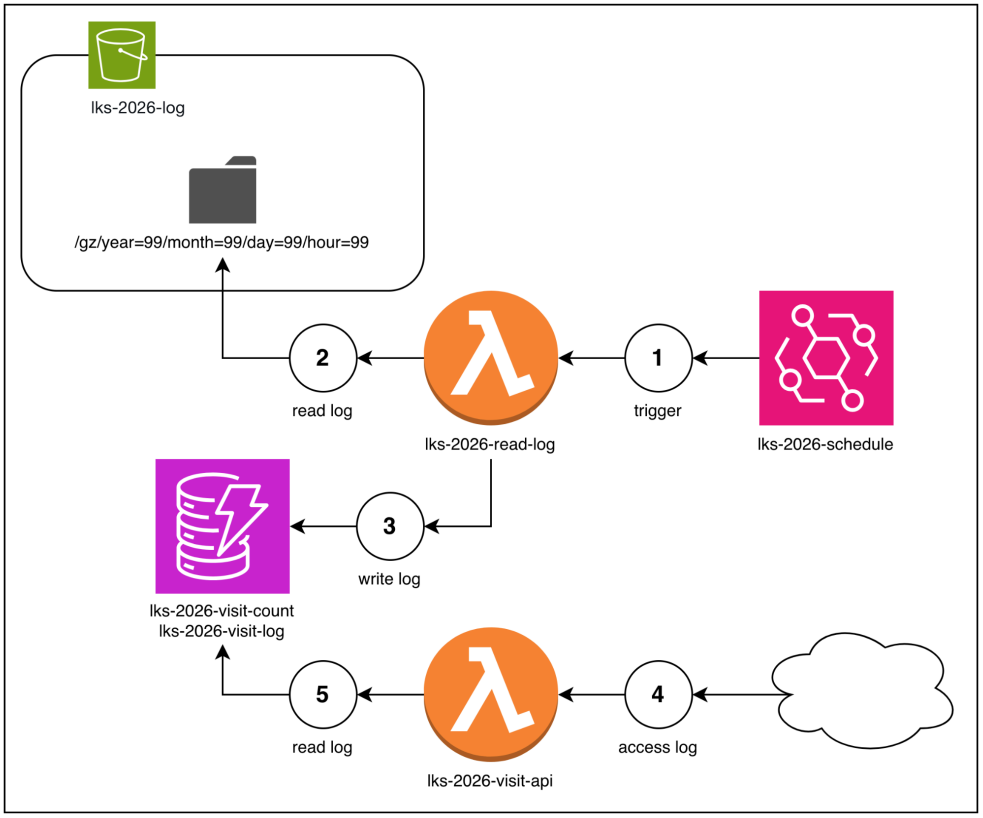
April 18, 2026

Topics 2	**DynamoDB, Lambda-Based Log Processing, EventBridge Scheduling, and Visit Count API Development**
Need	1. AWS Account with DynamoDB, Lambda, EventBridge, and IAM access
	2. Web Browser (Chrome / Firefox / etc)
	3. Code Editor (VSCode / Sublime / etc)
Duration	4 Hours

Prerequisites

This topic is a **continuation of Topic I**. All resources created in the previous topic — including the S3 static website bucket, S3 log bucket, CloudFront distribution, and Lambda function — are assumed to be available and functional.

Architecture Overview



Task

Important: All resources must be created in the **`ap-southeast-1`** (Singapore) region. Resources created outside the designated region will result in a significant score reduction.

Caution: Tagging each resource as specified is **mandatory**. A resource without the required tag will be considered as if it does not exist.

Part A — DynamoDB Table for Log Records

1. Create a **DynamoDB table** with the name **`lks-2026-visit-log`** to store CloudFront access log records
   - Partition key: **`request_uri`** (String)
   - Sort key: **`timestamp`** (String)
   - **Tag** the table with key `lks-2026-bks` and value `lks-2026-visit-log`
2. **Checkpoint I**: Confirm that the table status is **Active** before proceeding

Part B — DynamoDB Table for Visit Counter

1. Create a **DynamoDB table** with the name **`lks-2026-visit-count`** to store aggregated visit counts per URI
   - Partition key: **`request_uri`** (String)
   - **Tag** the table with key `lks-2026-bks` and value `lks-2026-visit-count`
2. **Checkpoint II**: Confirm that the table status is **Active** before proceeding

Part C — Lambda Function for Log Processing

1. Open the source file provided at the following URL and examine the code: **https://github.com/darkbcx/lks-2026-bks-topic-2/blob/main/lks-2026-read-log/lambda_function.py**
2. Determine the correct **runtime** by examining the source code
3. Create a **Lambda Function** with the name **`lks-2026-read-log`**
   - Architecture: **x86_64**
   - **Tag** the function with key `lks-2026-bks` and value `lks-2026-read-log`
4. **Copy the code** from the source file into the Lambda function editor
5. Add a **Layer** named **AWSSDKPandas** to the function — select the version that matches the runtime chosen in step 2
6. Set the function **timeout** to **1 minute** and **memory** to **1024 MB**
7. The Lambda function must have an **IAM role** (or inline policy) that grants the following permissions:
   On the S3 log bucket created in Topic I:
   - `s3:GetObject`

- `s3:ListBucket`

On the DynamoDB tables created in Part A and Part B:

- `dynamodb:BatchWriteItem`
- `dynamodb:UpdateItem`

8. Configure the following **environment variables**:

Variable	Value
`LOG_TABLE_NAME`	Name of the DynamoDB table created in **Part A**
`COUNT_TABLE_NAME`	Name of the DynamoDB table created in **Part B**
`S3_BUCKET_NAME`	Name of the S3 log bucket created in **Topic I**

9. **Checkpoint III**: Verify that the Lambda function is created successfully — confirm the runtime, environment variables, and IAM permissions are all correctly configured

Part D — EventBridge Scheduler

1. Create an **EventBridge Scheduler** with the following configuration:
   - Schedule name: **`lks-2026-schedule`**
   - Description: **`lks-2026-schedule`**
   - Schedule pattern: **Recurring schedule**
   - Time zone: **(UTC+07:00) Asia/Jakarta**
   - Schedule type: **Cron-based schedule**
   - Cron expression: the schedule must fire **every 1 hour**
   - Flexible time window: **5 minutes**
   - End date and time: **end of the current month**
2. Configure the target:
   - Target type: **Templated targets** → **AWS Lambda**
   - Target function: the Lambda function created in **Part C** (`lks-2026-read-log`)
3. Settings:
   - Action after schedule completion: `NONE`
   - Execution role: use the **default role** created by AWS for this scheduler
4. **Checkpoint IV**: Confirm that the schedule is in an **Active** state and the target Lambda function is correctly associated

Part E — Lambda API for Visit Counts

1. Download the zip file provided at the following URL: https://github.com/darkbcx/lks-2026-bks-topic-2/blob/main/lks-2026-visit-api/Archive.zip
2. Extract the zip file locally and examine the source code to determine the correct **runtime**
3. Create a **Lambda Function** with the name **`lks-2026-visit-api`**
   - Architecture: **x86_64**
   - **Tag** the function with key `lks-2026-bks` and value `lks-2026-visit-api`
4. Upload the zip file into the Lambda function
5. Set the function **timeout** to **1 minute** and **memory** to **1024 MB**
6. The Lambda function must have an **IAM role** (or inline policy) that grants the following permissions on the DynamoDB table created in **Part B**:
   - `dynamodb:GetItem`
   - `dynamodb:Scan`
   - `dynamodb:Query`
7. Configure the following **environment variable**:

Variable	Value
`COUNT_TABLE_NAME`	Name of the DynamoDB table created in **Part B**

8. Create a **Function URL** for this Lambda function with auth type **NONE** so it can be accessed publicly via HTTP
9. **Checkpoint V**: Open the Function URL root endpoint (`/`) in your browser and verify that it returns a valid JSON response

Part F — Final Verification

This is the final end-to-end test that validates the complete pipeline across **Topic I** and **Topic II**.

1. Empty the **S3 log bucket** created in Topic I — remove all existing log files
2. Delete all records from both DynamoDB tables created in Part A and Part B manually through the console so they start with a clean state
3. Open the **CloudFront distribution URL** (from Topic I) in your browser and navigate between pages multiple times to generate fresh access log entries
4. **Wait approximately 10–15 minutes** for CloudFront to deliver the new log files to the S3 log bucket

   Important: Ensure this waiting period and the next steps all occur **within the same hour** — the log processor groups records by hour
5. Manually trigger the `lks-2026-read-log` Lambda function (Part C) by running a **Test** event in the Lambda console — you may use any arbitrary JSON object as the test event (e.g. `{}`)

6. After the function executes successfully, open the **Function URL** of `lks-2026-visit-api` (Part E) and access the **`/visit-count`** endpoint in your browser
7. **Checkpoint VI**: Verify that the `/visit-count` endpoint returns JSON data containing visit counts that reflect the browsing activity from step 3
8. **Present the API response to the jury** as the final demonstration

---

Output

1. DynamoDB table `lks-2026-visit-log` is created with the correct partition key, sort key, and tag
2. DynamoDB table `lks-2026-visit-count` is created with the correct partition key and tag
3. Lambda function `lks-2026-read-log` is created with the correct runtime, IAM permissions, and environment variables
4. EventBridge Scheduler `lks-2026-schedule` is active with the correct cron expression and targets the `lks-2026-read-log` Lambda function
5. Lambda function `lks-2026-visit-api` is created with the correct runtime, IAM permissions, environment variable, and a Function URL that returns a valid JSON response
6. The complete pipeline works end-to-end: browsing the website generates logs, the Lambda processor writes records to DynamoDB, and the `/visit-count` API endpoint returns the aggregated visit data